

6주차 예습과제

📌 과제 종류	예습과제
≡ 범위	딥러닝 파이토치 교과서 6장 - Part 2 (6.1.5~)

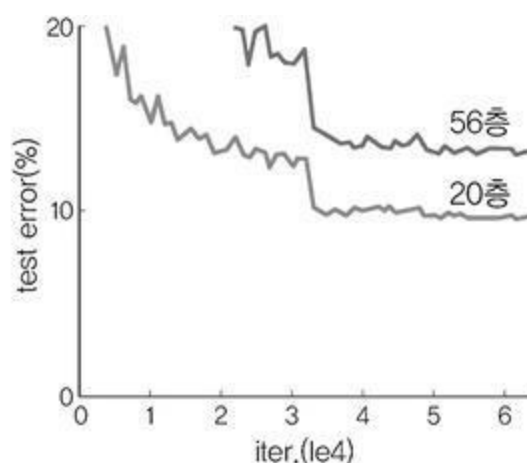
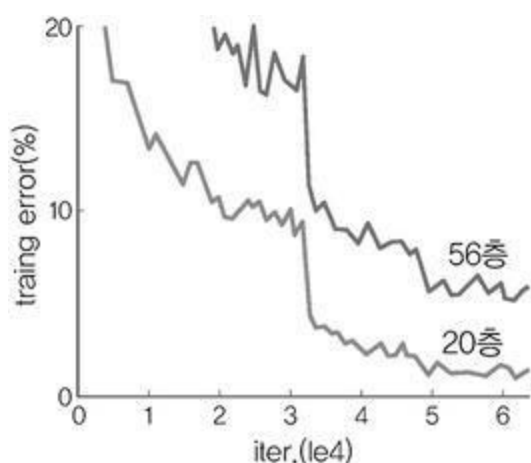
Chapter 06. 합성곱 신경망 II - part 2

6.1 이미지 분류를 위한 신경망

입력데이터가 '이미지'인 분류

6.1.5 ResNet

- 깊은 신경망을 효과적으로 학습하기 위한 방법으로 레지듀얼(residual, 잔여) 개념 고안
- 신경망이 깊이가 깊어질수록
 - 성능이 좋아지다가 일정 단계에 다다르면 성능이 나빠짐
 - 무조건 좋아지는 것은 아님

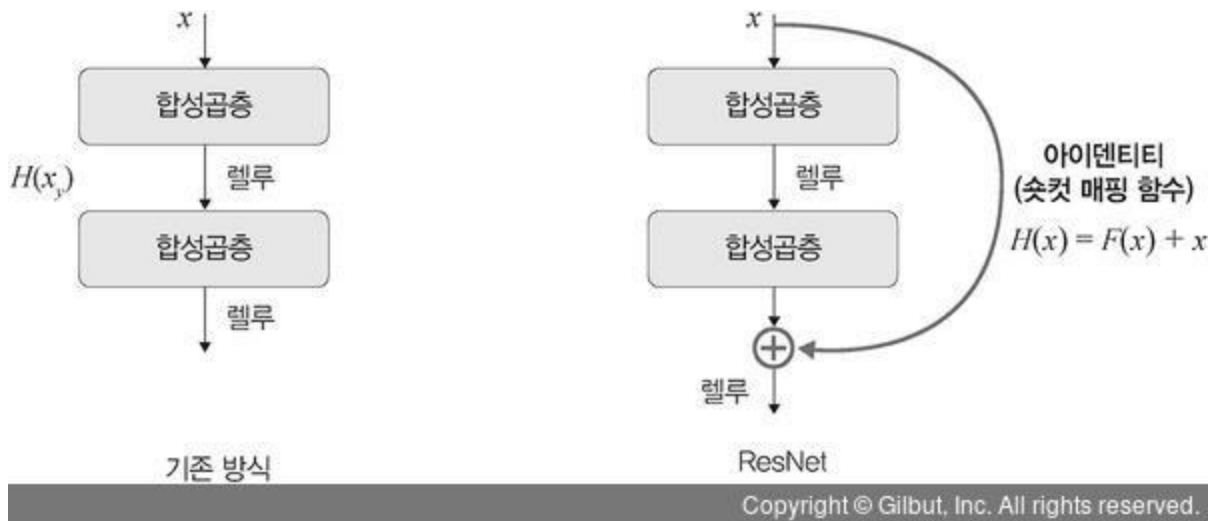


Copyright © Gilbut, Inc. All rights reserved.

- 네트워크 깊이가 깊다고 해서 무조건 성능이 좋아지는 것 X
→ ResNet : 문제 해결을 위해 **레지듀얼 블록(residual block)** 도입

레지듀얼 블록 : 기울기가 잘 전파될 수 있도록 일종의 숏컷(shortcut, skip connection)을 만들

- ResNet은 층이 총 152개 → **기울기 소멸 문제**가 발생할 수 있으므로 숏컷을 두어 기울기 소멸 문제를 방지함



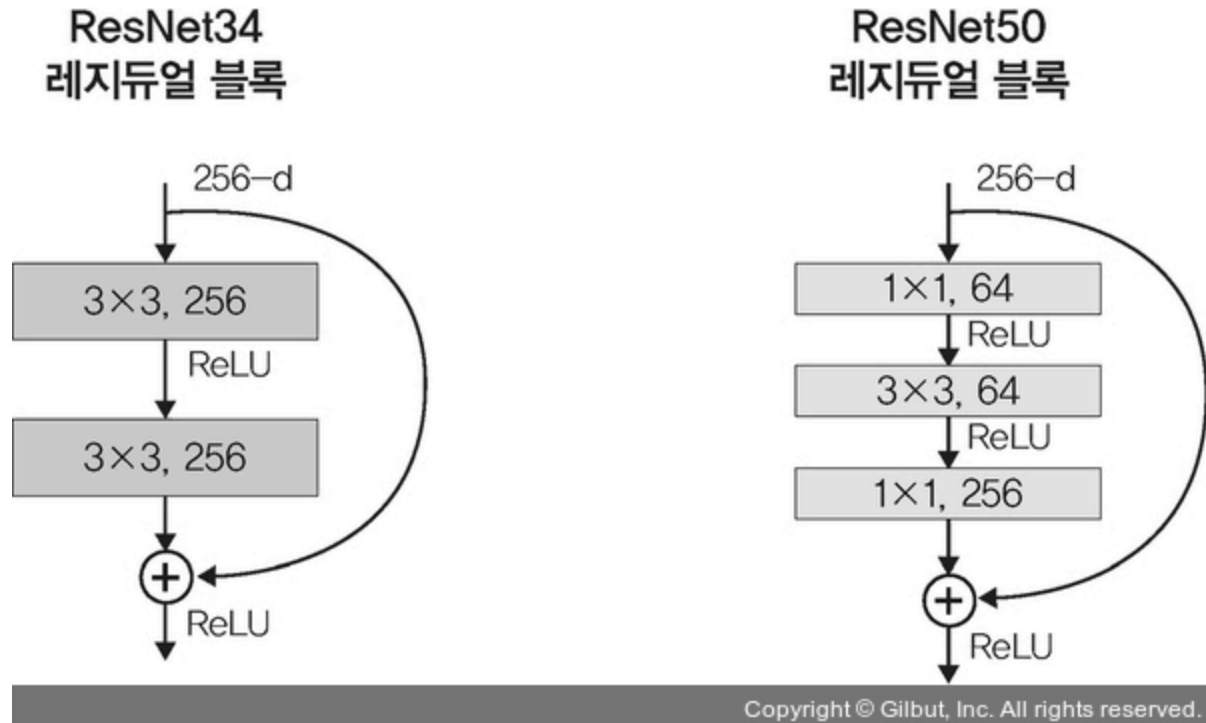
블록(block)

- 계층의 묶음. (합성곱층을 하나의 블록으로 묶음)
- 이렇게 묶인 계층들을 하나의 **레지듀얼 블록**이라고 함.
- 레지듀얼 블록을 여러 개 쌓은 것 → ResNet



- 계층을 계속 쌓아올리면 파라미터 수가 무제한으로 커짐
⇒ 병목 블록(bottleneck block)을 둬.

병목 블록(bottleneck block)

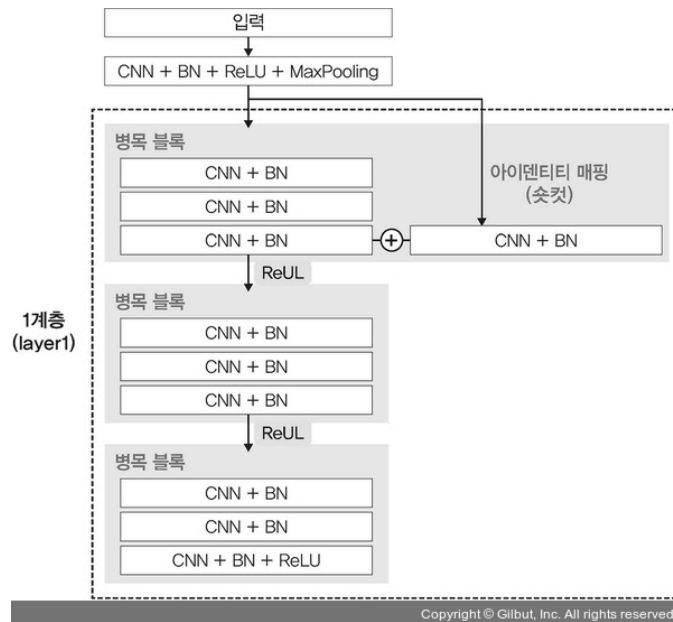


기본 블록과 병목 블록

- 병목 블록 사용: 파라미터 수가 감소하는 효과
- ResNet 34는 기본 블록, 50은 병목 블록 사용 → 깊이가 깊어져도 파라미터 수는 감소

아이덴티티 매핑 (숏컷, 스킵 연결)

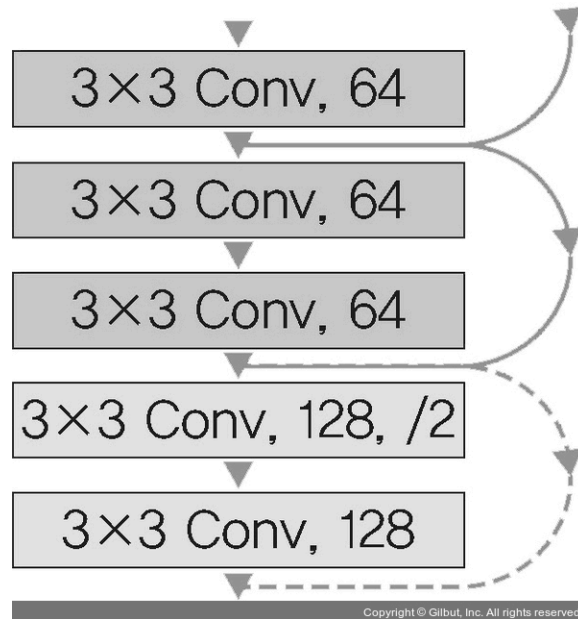
- 입력값x가 어떤 함수를 통과해도 다시 입력값x로 출력
- 입력값 = 출력값



```
def forward(self,x):
    i = x
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.conv2(x)
    x = self.bn2(x)
    if self.downsample is not None:
        i = self.downsample(i) #다운샘플 적용
    x += i #아이덴티티 매핑 적용
    x = self.relu(x)
    return x
```

다운샘플

- feature map 크기를 줄이기 위한 것, 풀링과 같은 역할



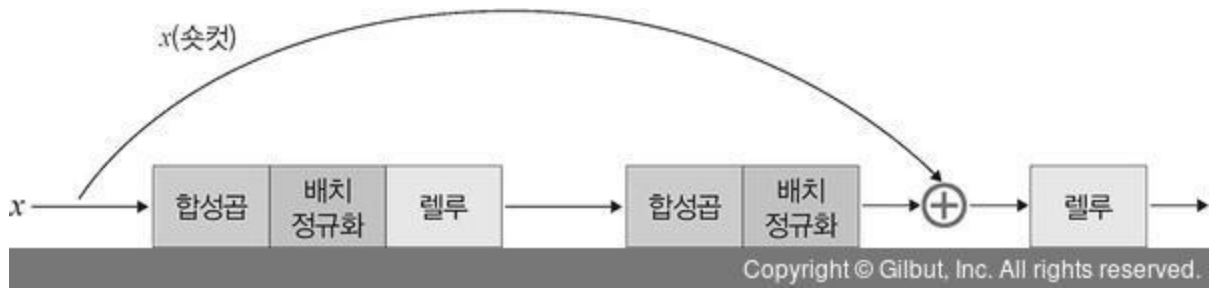
- 보라색 영역의 특성 맵 형태:
 - 첫 번째 블록에서 특성맵 형상 (28, 28, 64)
 - 세 번째 블록의 마지막 합성곱층과 아이덴티티 매핑(identity mapping)까지 완료된 후에도 (28, 28, 64)로 유지됨.
- 노란색 영역의 특성 맵 형태:
 - 시작 채널 수가 128로 늘어나고 /2가 있어 첫 번째 블록에서 합성곱층의 스트라이드가 2로 늘어났음을 의미 함. (14, 14, 128)로 바뀜.
 - 보라색 영역과 노란색 영역의 특성 맵 형태가 다르면 아이덴티티 매핑을 할 수 없으므로, 아이덴티티 매핑을 위해 **다운샘플(downsample)**이 필요함.

Projection Shortcut (프로젝션 숏컷, 합성곱 블록): 입력과 출력의 차원이 동일하지 않고, 입력의 차원을 출력에 맞춰 변경해야 하는 것

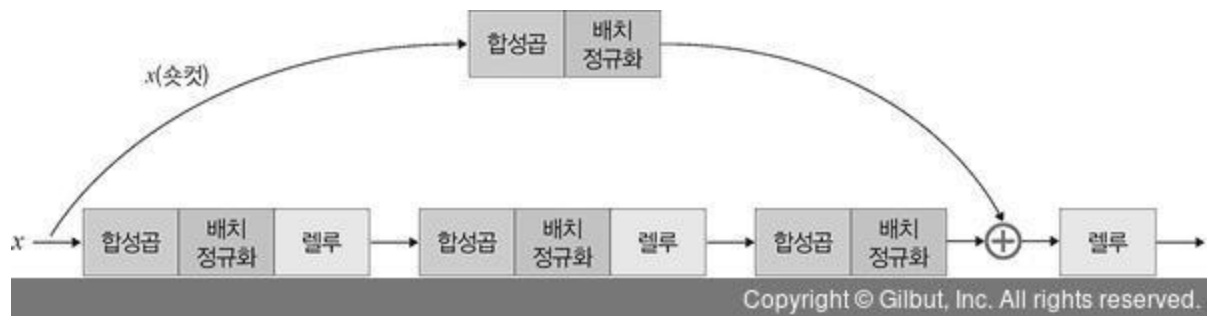
ResNet 구조

= 아이덴티티 블록(숏컷으로 만들어진 블록) + 합성곱 블록(합성곱층으로 구성됨)

- 아이덴티티 블록

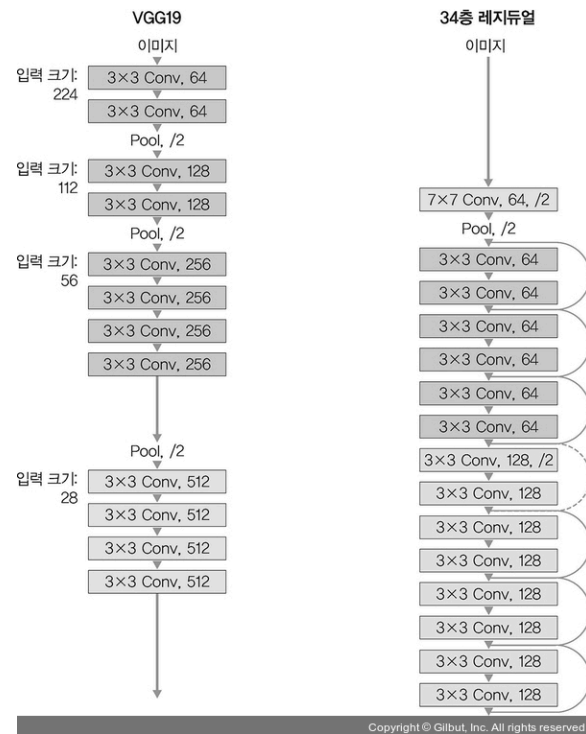


- 합성곱 블록



정리

- ResNet은 기본적으로 VGG19 구조를 뼈대로 하며, 거기에 합성곱층들을 추가해서 깊게 만든 후 숏컷들을 추가하는 것



6.2 객체 인식을 위한 신경망

객체 인식이란?

이미지나 영상 내에 있는 객체를 식별하는 컴퓨터 비전 기술

→ 이미지 · 영상 내에 있는 객체가 무엇인지 **분류**, 그 객체의 위치가 어디인지 박스로 나타내는 **위치 검출**

딥러닝을 이용한 객체 인식 알고리즘

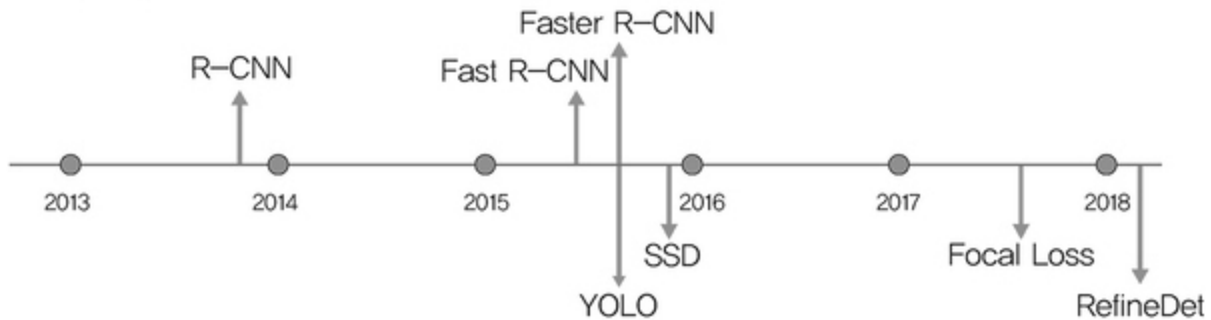
1단계 객체 인식

- 빠르지만 정확도가 낮음

2단계 객체 인식

- 느리지만 정확도가 높음
- YOLO계열, SSD계열

2단계 객체 인식



1단계 객체 인식

Copyright © Gilbut, Inc. All rights reserved.

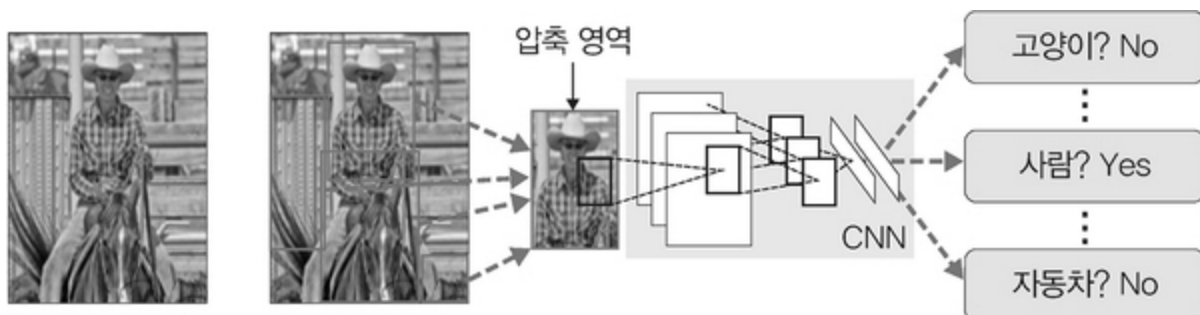
6.2.1 R-CNN

과거: 슬라이딩 윈도우

- 일정한 크기를 가지는 윈도우를 가지고 이미지의 모든 영역 탐색, 객체를 검출해 내는 방식
- 알고리즘의 비효율성 문제 → 현재는 선택적 탐색 알고리즘 적용한 후보 영역 많이 사용

• R-CNN

- 이미지 분류를 수행하는 CNN과 이미지에서 객체가 있을 만한 영역을 제안해 주는 후보 영역 알고리즘을 결합한 알고리즘



1 이미지 삽입 2 후보 영역 추출 3 CNN 특성 계산 4 영역 분류

Copyright © Gilbut, Inc. All rights reserved.

1. 이미지를 입력으로 받습니다.
2. 2000개의 바운딩 박스(bounding box)를 선택적 탐색 알고리즘으로 추출한 후 잘라 내고(cropping), CNN 모델에 넣기 위해 같은 크기(227×227 픽셀)로 통일합니다(warping).
3. 크기가 동일한 이미지 2000개에 각각 CNN 모델을 적용합니다.
4. 각각 분류를 진행하여 결과를 도출합니다.

선택적 탐색

- 객체 인식이나 검출을 위한 가능한 후보 영역을 알아내는 방법
- 분할 방식을 이용하여 시드를 선정, 그 시드에 대한 완전 탐색 적용
- 선택적 탐색의 세 단계 과정
 1. 초기 영역 생성 : 입력된 이미지를 영역 다수개로 분할
 2. 작은 영역의 통합 : 1단계에서 나눈 영역들을 비슷한 영역으로 통합함(greedy 알고리즘 사용. 비슷한 영역이 하나로 통합될 때까지 반복)
 3. 후보 영역 생성 : 통합된 영역을 기반으로 후보 영역(바운딩 박스)를 추출함

용어 정리

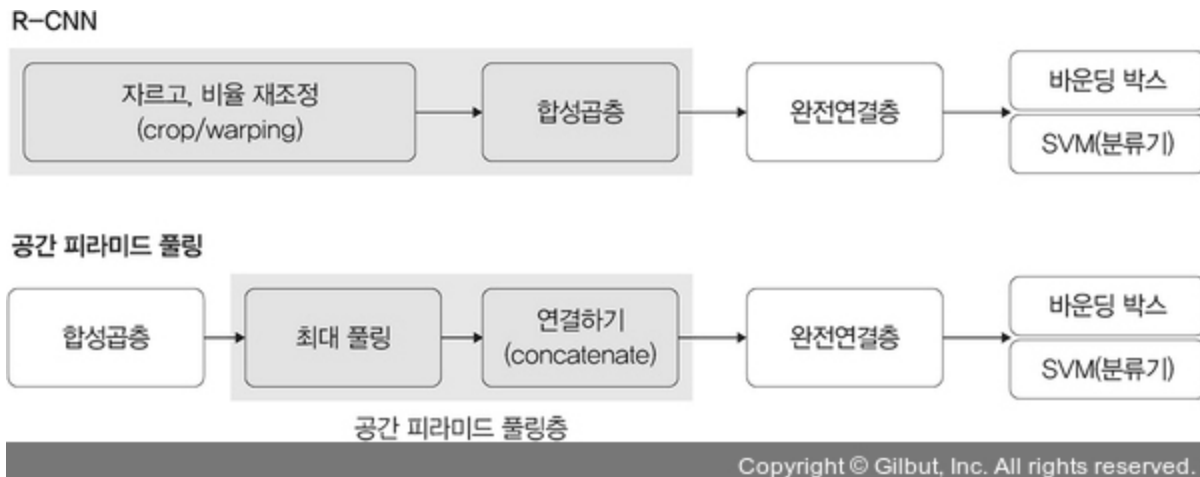
- **완전 탐색**: 후보가 될 만한 대상의 다른 상황(크기 · 비율)을 고려해서 후보 영역을 찾는 기법
- **분할**: 영상 데이터의 특성(색상, 모양, 무늬 등)에 따라 분할하여 후보 영역을 선정하는 기법
- **후보 영역(바운딩 박스)**: 3D 객체의 형태를 모두 포함할 수 있는 최소 크기의 박스
- **시드**: 영상에서는 특정 기준점의 픽셀에서 점점 의미가 같은 영상 범위까지 픽셀을 확장해 나가면서 분할함. 이때 특정 기준점이 되는 픽셀
- **그리디 알고리즘**: 여러 가지 경우 중 하나를 결정해야 할 때마다 그 순간에 최적이라고 생각되는 것을 선택해 나가는 방식
- **R-CNN의 단점**

1. 1. 세 단계의 복잡한 학습 과정
2. 긴 학습시간과 대용량 저장 공간 필요
3. 느린 객체 검출 속도

⇒ 해결 : **Fast R-CNN**

6.2.2 공간 피라미드 풀링

- 기존 CNN구조에서 발생하는 신경망 통과 시 이미지를 자르거나 비율을 조정하면 물체의 일부분이 잘려나가 본래의 생김새와 달라지는 문제점 해결 위해 등장



- 공간 피라미드 풀링
 - 입력 이미지의 크기에 관계없이 합성곱층 통과, 완전연결층에 전달되기 전에 특성 맵들을 동일한 크기로 조절해주는 풀링층 적용 기법
 - 입력 이미지의 크기를 조절하지 않고 합성곱층을 통과시키기 때문에 원본 이미지의 특징이 훼손되지 않는 특성 맵을 얻을 수 있음
 - 이미지 분류나 객체 인식 같은 여러 작업에 적용할 수 있다는 장점

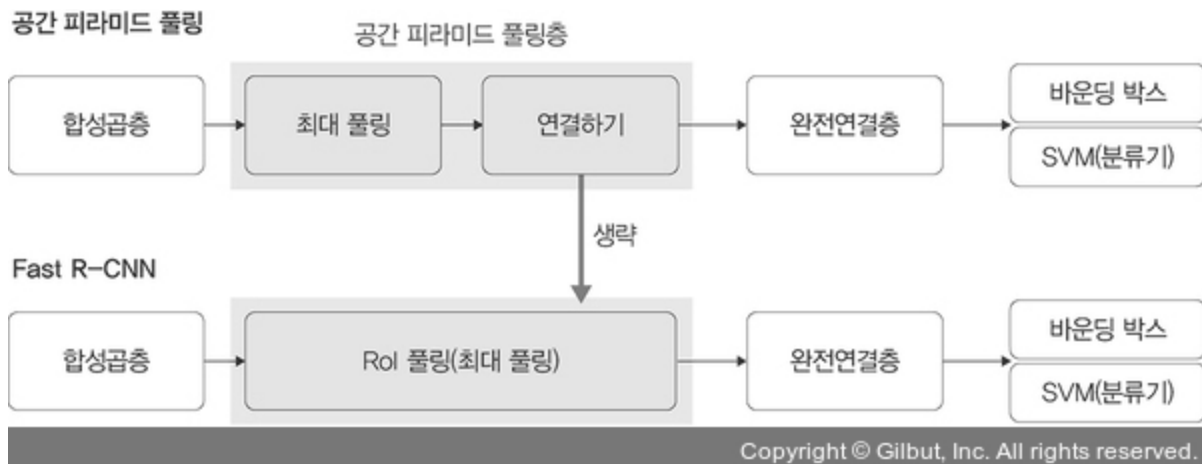
6.2.3 Fast R-CNN

- R-CNN의 속도 문제 개선 위해 등장

- 바운딩 박스마다 CNN을 돌림, 분류를 위한 긴 학습시간

Fast R-CNN

- 속도 문제를 개선하기 위해 **RoI 풀링** 도입
- 선택적 탐색에서 찾은 바운딩 박스 정보가 CNN을 통과하면서 유지되도록 함
- 최종 CNN 특성 맵은 풀링을 적용하여 완전연결층을 통과하도록 크기 조정
→ 바운딩 박스마다 CNN 돌리는 시간 단축



RoI 풀링

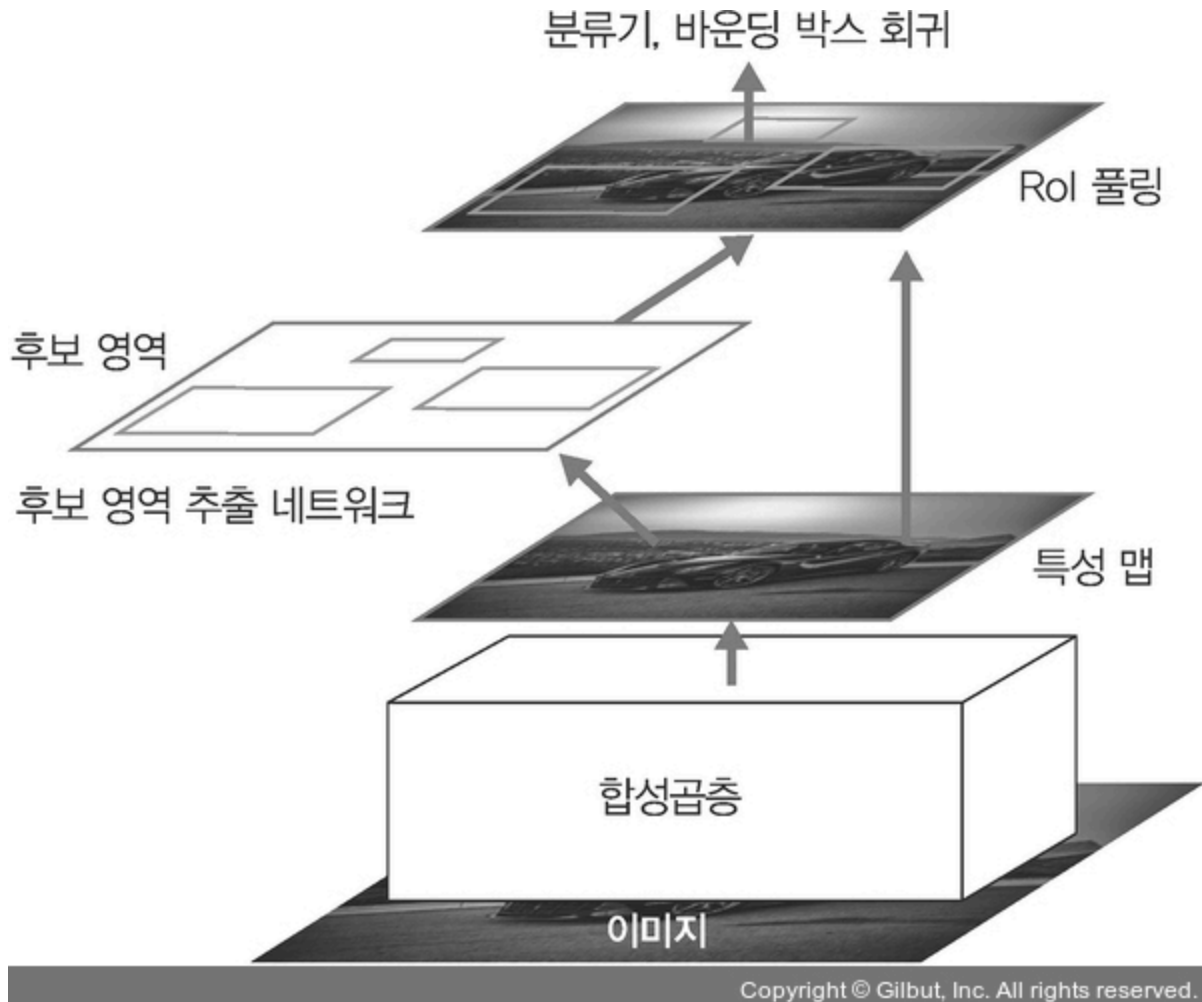
- 크기가 다른 특성 맵의 영역마다 스트라이드를 다르게 최대 풀링을 적용하여 결괏값 크기를 동일하게 맞추는 방법

6.2.4 Faster R-CNN

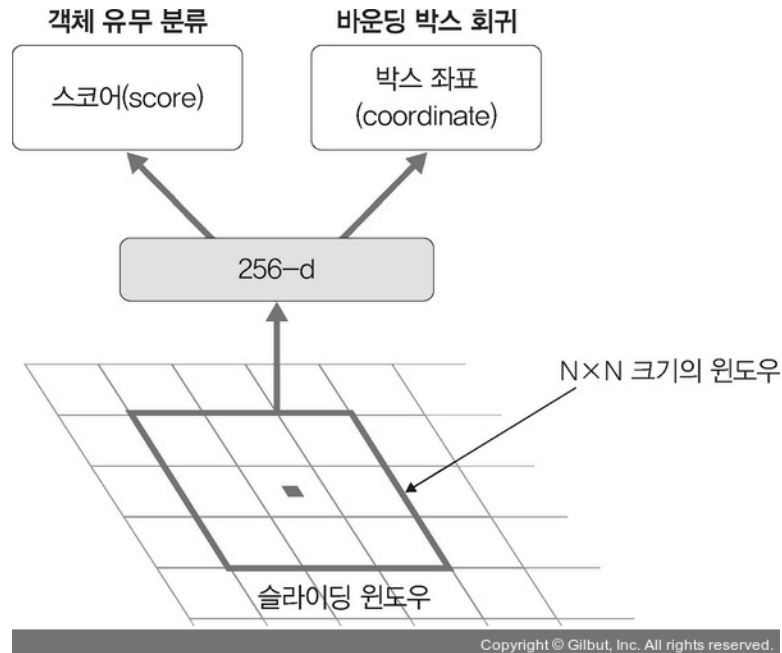
Faster R-CNN

- '더욱 빠른' 객체 인식을 수행하기 위한 네트워크
- 후보 영역 생성을 CNN 내부 네트워크에서 진행할 수 있도록 설계
- 외부의 느린 선택적 탐색(CPU로 계산) 대신 내부의 빠른 **RPN(GPU로 계산)**을 사용

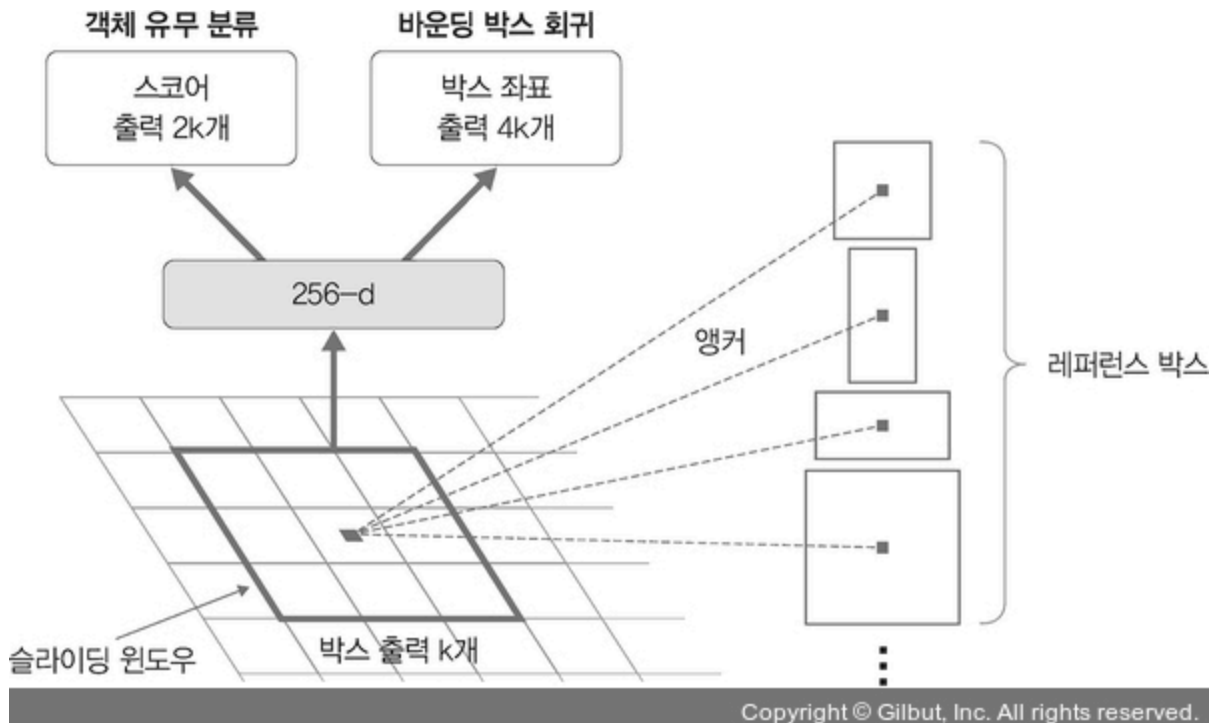
- RPN은 마지막 합성곱층 다음에 위치, 그 뒤에 RoI풀링, 분류기, 바운딩 박스 회귀 위치



- 후보 영역 추출 네트워크
 - 특성 맵에서 $N \times N$ 크기의 슬라이딩 윈도우 영역을 입력으로 받음
 - 해당 영역에 객체 존재 유무를 판단하기 위해 이진 분류를 수행
 - 바운딩 박스 회귀를 통해 위치 보정(좌표점 추론) 수행
 - 슬라이딩 윈도우 방식으로 특성 맵 전체를 탐색하며 객체 검출



- 단점: 다양한 크기와 비율의 이미지 수용이 어려움 → 이를 보완하기 위해 앵커 방식 사용
- 후보 영역 추출 네트워크의 출력 값은 모든 앵커 위치에 대해 각 객체와 배경을 판단하는 2k개의 분류에 대한 출력, x,y,w,h 위치 보정 값을 위한 4k개의 회귀 출력 가짐
- 특성 맵 크기가 $w \times h$ 라면 하나의 특성 맵에 앵커가 총 $w \times h \times k$ 개 존재



6.3. 이미지 분할을 위한 신경망

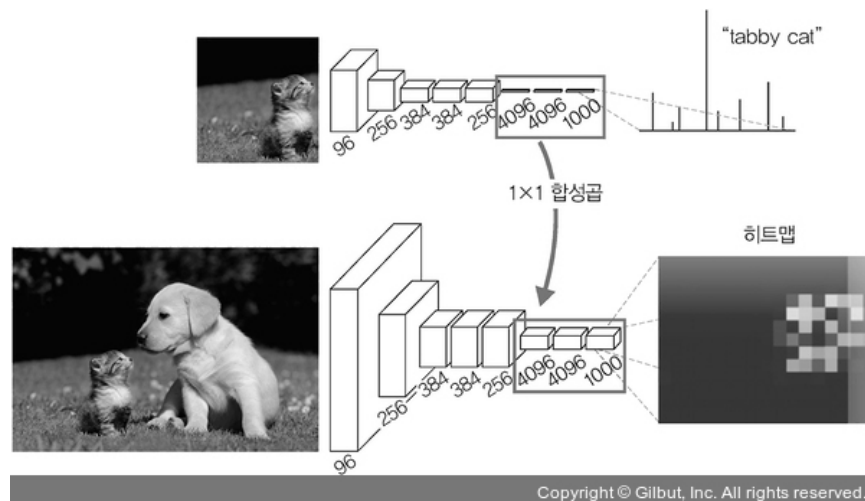
| 이미지 분할

- 신경망을 훈련시켜 이미지를 픽셀 단위로 분할하고, 객체 추출

6.3.1 완전 합성곱 네트워크

| 완전 합성곱 네트워크

- 완전연결층을 1x1 합성곱으로 대체하는 것
- 이미지 분류에서 우수한 성능을 보인 CNN 기반 모델(AlexNet, VGG16, GoogLeNet)을 변형시켜 이미지 분할에 적합하도록 만든 네트워크



6.3.2 합성곱 & 역합성곱 네트워크

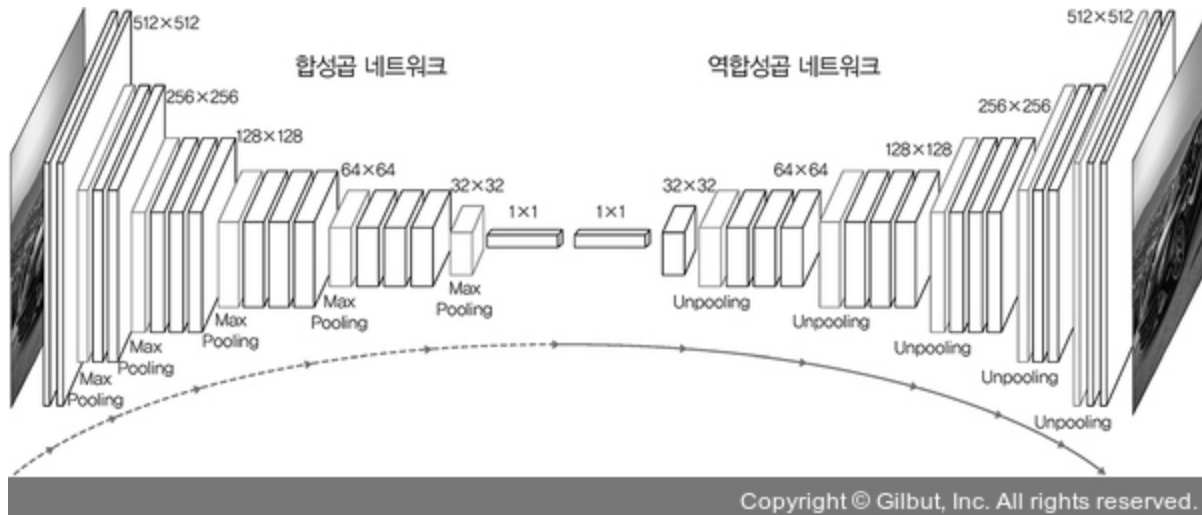
완전 합성곱 네트워크 단점

- 여러 단계의 합성곱층과 풀링층을 거치면서 해상도가 낮아짐
- 낮아진 해상도를 복원하기 위해 업 샘플링 방식을 사용하기 때문에 이미지의 세부 정보들을 잃어버리는 문제 발생

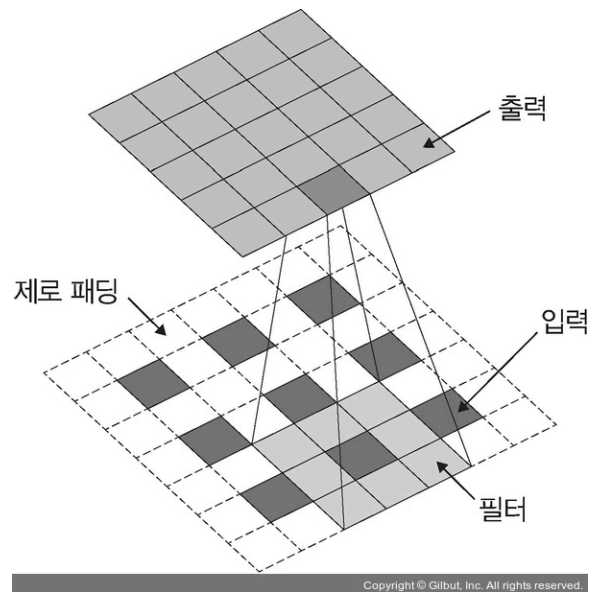
→ 역합성곱 네트워크 도입

역합성곱

- CNN의 최종 출력 결과를 원래의 이미지와 같은 크기로 만들고 싶을 때 사용
- 시멘틱 분할 등에 활용
- 업샘플링(upsampling)이라고 함.



- 동작
 1. 각각의 픽셀 주위에 제로 패딩을 추가
 2. 이렇게 패딩된 것에 합성곱 연산을 수행함



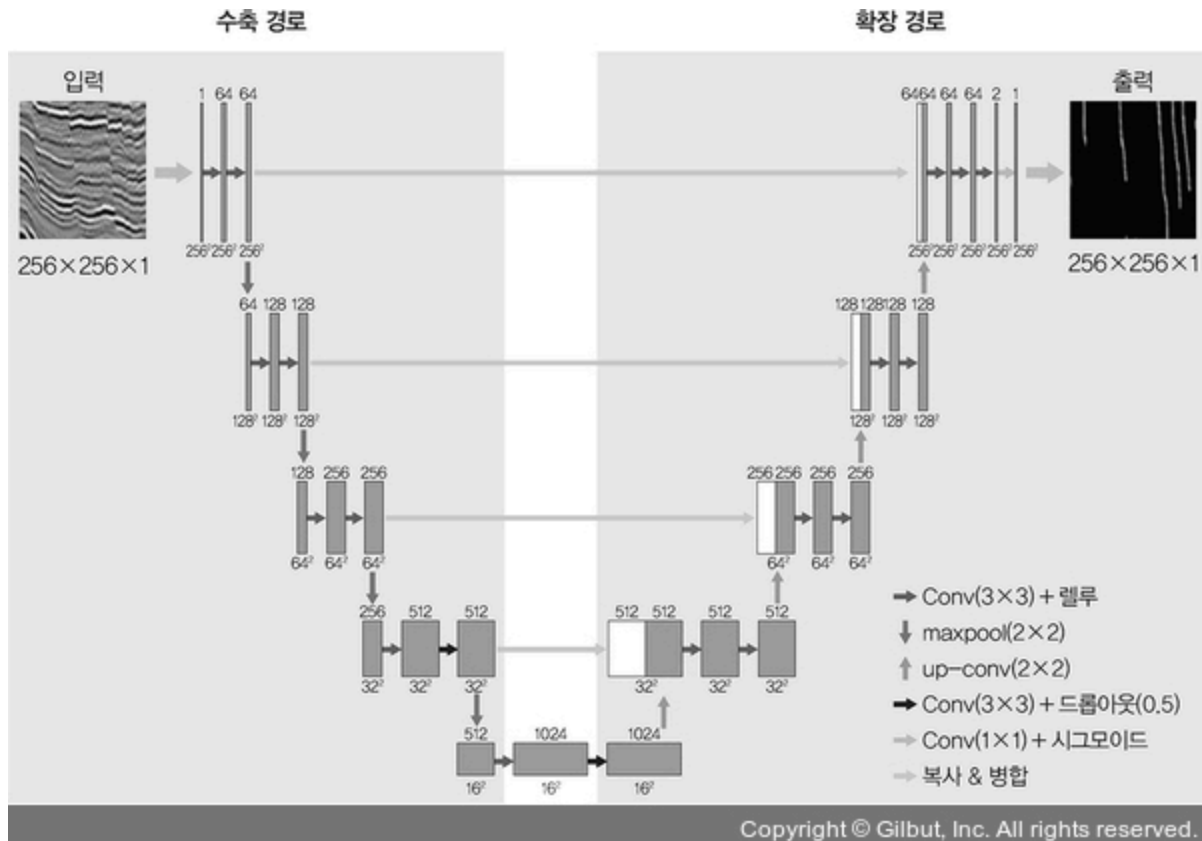
6.3.3 U-Net

U-Net

- 바이오 메디컬 이미지 분할을 위한 합성곱 신경망
- 특징
 - 속도가 빠름 : 이미 검증이 끝난 패치는 건너뛰기 때문
 - 트레이드오프(trade-off)에 빠지지 않음: 컨텍스트 인식과 지역화 트레이드오프 문제 개선

U-Net의 구조

- FCN 기반으로 구축되었으며, 수축 경로와 확장 경로로 구성
 - **수축 경로**: 컨텍스트 포착
 - **확장 경로**: 특성 맵을 업 샘플링하고, 수축 경로에서 포착한 특성 맵의 컨텍스트와 결합하여 정확한 지역화를 수행
- U-Net은 3×3 합성곱이 주를 이루는데 각 합성곱 블록은 3×3 합성곱 두 개로 구성되어 있으며, 그 사이에 드롭아웃(dropout)이 있음



→ 크기가 다양한 이미지의 객체를 분할하기 위해 크기가 다양한 특성 맵을 병합할 수 있도록 다
운 샘플링, 업 샘플링 순서대로 반복하는 구조

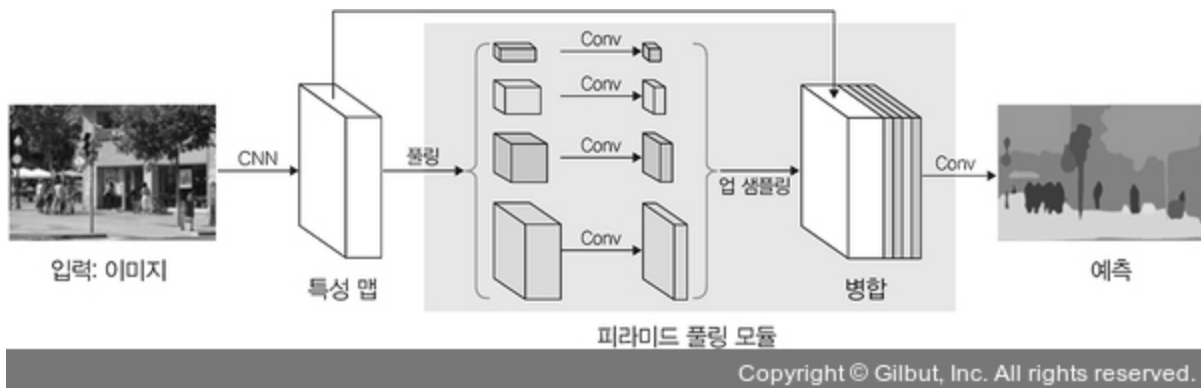
6.3.4 PSPNet

PSPNet

- 완전연결층의 한계를 극복하기 위해 피라미드 풀링 모듈 추가

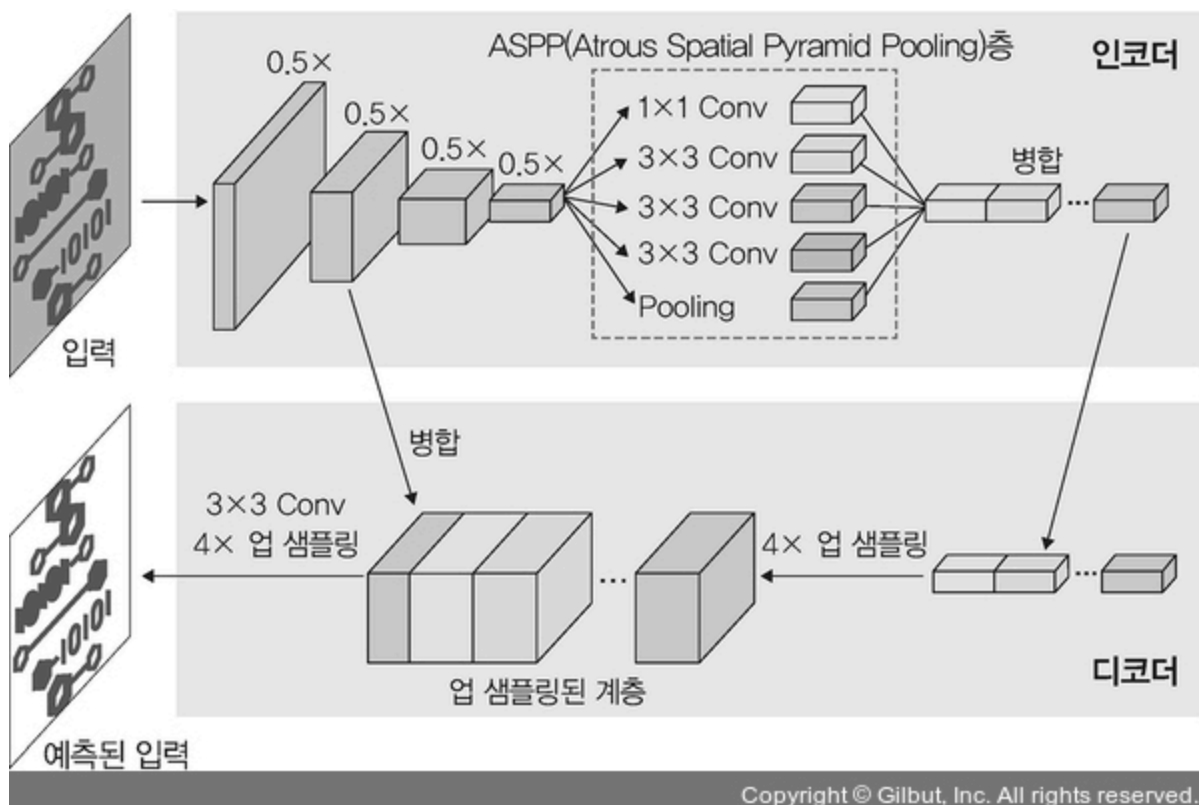
훈련 과정

1. 다양한 크기(1×1, 2×2, 3×3)로 풀링하여 서로 다른 범위의 정보 추출
2. 1×1 합성곱으로 채널 수를 $\text{입력 채널 수} / N$ (풀링층 개수)으로 조정
3. 모듈의 입력 크기에 맞게 특성 맵을 원래 크기로 업샘플링 (양선형 보간법 사용)
4. 원래 특성 맵과 새로운 특성 맵들을 병합



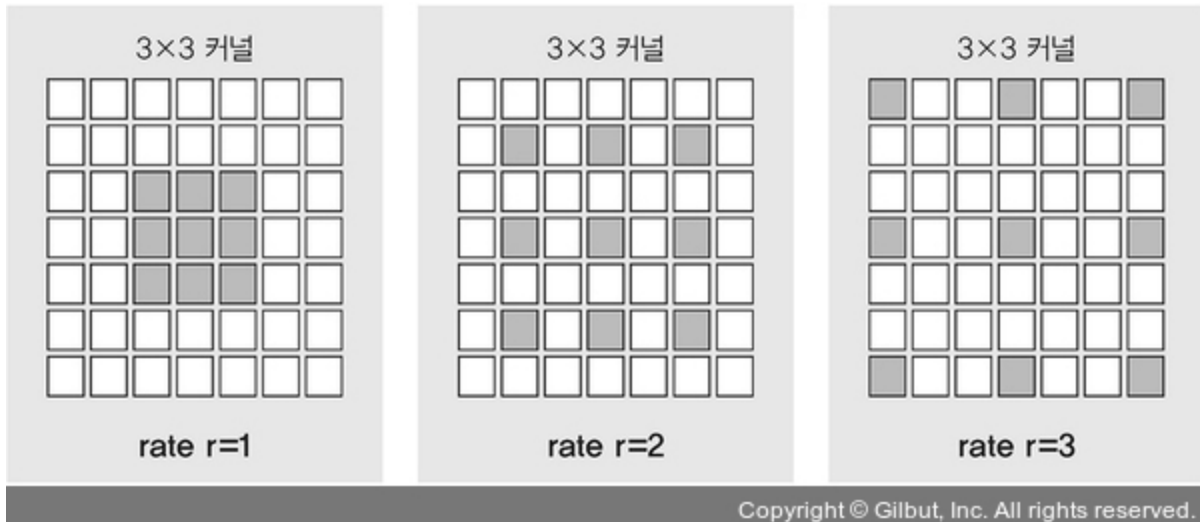
6.3.5 DeepLabv3/DeepLabv3+

- 완전연결층의 단점을 보완하기 위해 Atrous 합성곱을 사용하는 네트워크
- 인코더 디코더 구조



Atrous 합성곱

- 필터 내부에 빈 공간을 둔 채 동작
- 파라미터 rate: 빈 공간의 정도를 결정하며, rate r 값이 커질수록 빈 공간이 더 많아짐



- 이미지 분할에서 중요한 수용 영역은 Atrous 합성곱을 활용하면 파라미터 수를 늘리지 않고도 수용 영역을 키울 수 있음.